DAN'S GAMING LEAGUE SERIES (DGLS)

WEB APPLICATION ENGINEERING BLUEPRINT & STRATEGIC ROADMAP

1. Executive Summary & Core Paradigm Shift

The Dan's Gaming League Series (DGLS) is currently in its third season of competitive 2v2 Counter-Strike Wingman action. Historically managed across a series of interconnected spreadsheets, the league requires a dedicated web application platform to support its scale, enhance transparency, and eliminate formula maintenance failures (such as Excel `#N/A` errors).

Based on structural reviews of past tournament assets, DGLS implements a specialized **individual rotating mixer format**. There are no persistent teams. Instead, players are registered as independent entities and are dynamically paired into opposing factions (historically designated as "Shirts" and "Skins") alongside rotating weekly byes (e.g., accommodating 13 players in Season 3).

> **Strategic Priority Realignment:** To optimize engineering velocity for the active Season 3 deployment, interactive in-app manual match reporting forms have been down-prioritized. The core operational loop will continue to use the current manual spreadsheet architecture for match-night data entry. The web platform will operate as an advanced read-and-aggregation platform, focusing engineering capital on a high-fidelity frontend UI/UX presentation layer and a completely automated custom playoff state machine.

2. Relational Database Architecture (PostgreSQL)

To cleanly represent rotating combinations, historic season versions, and individual combat metrics, a highly normalized relational database schema is deployed. This handles variations across seasons, such as Season 1's 9-round target win condition versus Season 2 and 3's 13-round target win conditions.

```
-- Enumerations for Strict State Validation
CREATE TYPE season_status AS ENUM ('ARCHIVED', 'ACTIVE', 'UPCOMING');
CREATE TYPE faction_side AS ENUM ('SHIRTS', 'SKINS');

-- Seasons Configuration Profile
CREATE TABLE seasons (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,          -- 'Season 1', 'Season 2', 'Season 3'
    status season_status NOT NULL DEFAULT 'UPCOMING',
    target_win_rounds INT NOT NULL DEFAULT 13, -- S1 = 9, S2/S3 = 13
    buy_in_amount NUMERIC(5,2) DEFAULT 10.00
);

-- Master Player Directory
CREATE TABLE players (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,          -- Core roster mapping ('Adam', 'Zach', etc.)
    discord_id VARCHAR(30) UNIQUE
);

-- Schedule Structural Framework Tracking Weekly Byes
CREATE TABLE weeks (
    id SERIAL PRIMARY KEY,
    season_id INT REFERENCES seasons(id) ON DELETE CASCADE,
```

```
    week_number INT NOT NULL,
    bye_player_id INT REFERENCES players(id),  -- Explicitly handles odd-numbered player counts
    UNIQUE (season_id, week_number)
);

-- Core Match Node
CREATE TABLE matches (
    id SERIAL PRIMARY KEY,
    week_id INT REFERENCES weeks(id) ON DELETE CASCADE,
    match_number INT NOT NULL,                -- Sequential game identifier per week
    final_score VARCHAR(10),                  -- '13-7', '9-13'
    picked_map VARCHAR(50),
    shirts_ban VARCHAR(50),
    skins_ban1 VARCHAR(50),
    skins_ban2 VARCHAR(50),
    shirts_pick VARCHAR(50),
    skins_starting_side VARCHAR(5),           -- 'CT' or 'T'
    is_playoff_game BOOLEAN DEFAULT FALSE,
    is_interpolated BOOLEAN DEFAULT FALSE,    -- Flags stats adjusted due to server drops
    notes TEXT
);

-- Atomic Player-Match Log (Core Source of Truth)
CREATE TABLE player_match_stats (
    id SERIAL PRIMARY KEY,
    match_id INT REFERENCES matches(id) ON DELETE CASCADE,
    player_id INT REFERENCES players(id) ON DELETE CASCADE,
    faction faction_side NOT NULL,
    kills INT NOT NULL DEFAULT 0,
    assists INT NOT NULL DEFAULT 0,
    deaths INT NOT NULL DEFAULT 0,
    adr INT NOT NULL DEFAULT 0,
    damage INT NOT NULL DEFAULT 0,
    rounds_played INT NOT NULL DEFAULT 0,
    rounds_won INT NOT NULL DEFAULT 0,
    is_win BOOLEAN NOT NULL DEFAULT FALSE,
    UNIQUE (match_id, player_id)
);
```

3. Spreadsheet Ingestion & Synchronization Flow

Because data entry remains inside the spreadsheets, the platform reads structured exports from the Season 3 Regular Season tab. The ingestion pipeline converts multi-row visual block matrix layouts into rows within the database.

The Ingestion Parsing Ruleset:

1. **Week & Bye Demarcation:** The parser loops through rows searching for headers containing `Week [X]` and `BYE: [PlayerName]`. When detected, a new record is appended to the `weeks` table, locking down the bye player's id.
2. **Match Header Boundaries:** The presence of a match index number combined with a score string (e.g., `13-5`) opens a new match record contextualized under the active week node.
3. **Faction Ingestion & Splitting:** The engine processes the block horizontally. It reads rows containing `Shirts` and `Skins`, map pick/ban data strings (e.g., `Shirts Ban`, `Skins Ban 1`, `Skins Ban 2`, `Shirts Pick`, `Skins Side`), and parses individual rows down into four separate `player_match_stats` records.

4. Analytics Pipeline & Individual Standings Metrics

Sorting rankings directly by total wins introduces competitive bias in individual-mixer formats where players maintain uneven matches played due to the scheduling of bye weeks. The core analytical layer bypasses static counts to calculate contextual ratios dynamically.

Leaderboard Sorting Hierarchy Matrix

Rank Priority	Statistical Metric	Mathematical Formula	Analytical Purpose
1. Primary	Win Percentage (Win %)	$W_p = rac{\Sigma Wins}{\Sigma Games} imes 100$	Normalizes leaderboard positions across players with scheduled byes.
2. Secondary	Round Win Rate (RWR %)	$R_{wr} = rac{\Sigma Rounds_{Won}}{\Sigma Rounds_{Played}} imes 100$	Measures round-level efficiency; serves as primary tie-breaker.
3. Tertiary	Kill/Death Ratio (K/D)	$K/D = rac{\Sigma Kills}{\Sigma Deaths}$	Evaluates individual gunfight performance.
4. Quaternary	Average Damage / Round (ADR)	$ADR = rac{\Sigma Damage}{\Sigma Rounds_{Played}}$	Determines true map impact independent of last-hit kills.

The platform executes these calculations over raw table values through a highly indexed PostgreSQL database view, ensuring rapid load times for dashboard leaderboards:

```
CREATE OR REPLACE VIEW view_player_standings AS
SELECT
    w.season_id,
    p.id AS player_id,
    p.name,
    COUNT(pms.id) AS matches_played,
    COUNT(CASE WHEN pms.is_win THEN 1 END) AS wins,
    COUNT(CASE WHEN NOT pms.is_win THEN 1 END) AS losses,
    ROUND((COUNT(CASE WHEN pms.is_win THEN 1 END)::NUMERIC / NULLIF(COUNT(pms.id), 0)) * 100, 2)
AS win_rate,
    ROUND((SUM(pms.rounds_won)::NUMERIC / NULLIF(SUM(pms.rounds_played), 0)) * 100, 2) AS
```

```
round_win_rate,
    ROUND(SUM(pms.kills)::NUMERIC / NULLIF(SUM(pms.deaths), 0), 2) AS kd_ratio,
    ROUND(AVG(pms.adr), 1) AS avg_adr
FROM players p
JOIN player_match_stats pms ON p.id = pms.player_id
JOIN matches m ON pms.match_id = m.id
JOIN weeks w ON m.week_id = w.id
GROUP BY w.season_id, p.id, p.name;
```

5. Frontend UI/UX Experience Specification

The visual architecture focuses on data transparency and community culture. Rather than generic layout styles, the interface surfaces the performance data that drives the league's banter.

The Chemistry Matrix Layout

A sortable, interactive grid plotting player-to-player relationships. Users select a player profile to unlock:

- **"Synergy Index" (With):** Displays win percentages when partnered with every other league entity. Instantly identifies who provides the ultimate carrying assistance.
- **"Nemesis Index" (Against):** Dissects win rates when playing against specific competitors, uncovering systemic structural blockades in performance.

The Historical Vault Dropdown

Provides retroactive access to archived spreadsheets. Selecting Season 1 or Season 2 updates the dashboard state instantly:

- Filters queries by historical season IDs.
- Displays different target score limits (e.g., highlighting Season 1's 9-round maps).
- Surfaces contextual annotations (such as historic "wacky lag" server disruptions notes).

Wacky Awards & Analytics Ticker

Tracks high and low performance statistical extremes across the regular season:

- 👑 **The Hard Carry:** Most damage dealt in a single match block across the league.
- 🩹 **The Sacrificial Lamb:** Highest deaths per round average (DPR) on successful winning match sets.
- 📈 **Schedule Difficulty Index:** Pulls values calculated in the spreadsheet's `Schedule Difficulty Calculator` tab to overlay a Normalized Difficulty factor (e.g., *Eric: 100% Difficulty* vs *Andrew: 69% Difficulty*) over standard leaderboard rankings.

6. Custom Season 3 Playoff Engine

The Season 3 Playoff configuration features a highly complex multi-tiered design that cannot be represented by generic tournament software tools. The playoff engine acts as a localized state machine that transitions through distinct stages once the regular season records lock down.

Tier 1: The Play-In Elimination Bracket

The regular season standings isolate the bottom four positions (**Seeds 10, 11, 12, and 13**). The automated engine freezes these entities into a single qualification node:

- **Match Assignments:** The engine structures a 2-round matchup matrix based on original seeding (e.g., `Seed 10 vs Seed 11` and `Seed 12 vs Seed 13`).
- **The Squeak-In Clause:** Player records are tracked through this mini-bracket. The unique individual who achieves a flawless 2-0 match record instantly secures the final playoff slot, while the remaining three components are locked out into permanent bottom standings.

Tier 2: The Dual Group Stage & Randomized Seeding Pool

Once the Play-In qualifier is determined, the tournament state shifts to a multi-pool group structure:

- **The Seed 1 Absolute Exception:** The regular season champion (Seed 1) bypasses the Group Stage entirely. The engine locks their record and places them directly into the Grand Finals node as a waiting finalist.
- **Seeding Pool Sorting Strategy:** The remaining nine competitors are split into distinct athletic selection brackets:
  - *Pool S:* Seeds 2, 3, 4, and 5
  - *Pool Z:* Seeds 6, 7, 8, 9, plus the unique Play-In Bracket Winner
- **Random Balancing Shuffle:** The engine executes a randomized assignment script that extracts exactly 2 entities from Pool S and 2 entities from Pool Z to construct **Group 1**. The remaining components are routed to **Group 2**. This ensures a balanced skill distribution across both groups.
- **Group Play Rules & Tie-Breakers:** Each group plays isolated round-robin sets. The engine evaluates standings within each cell. If records tie, the system evaluates the **Round Win Rate (RWR %)** earned during the tournament stage to establish group hierarchy.

Tier 3: The Grand Finals

The top performers from Group 1 and Group 2 advance to face Seed 1 in the final phase. The system generates the final bracket structures and records the grand final series results, crowning the DGLS Season 3 Champion.

7. Tactical Implementation Roadmap

> **Milestone 1: Database Setup & Historic Data Migration (Weeks 1-2)**
> Provision the core PostgreSQL instance. Write an isolated Node.js script to read exported CSV dumps of Season 1 and Season 2 datasets. Parse, transform, and load historical records into the tables to ensure the archive views work on day one.

Milestone 2: Spreadsheet Ingestion Sync Pipeline (Weeks 3-4)
Build a secure file ingestion dashboard endpoint. Code the parsing logic to decode the unique multi-row game blocks from the live Season 3 spreadsheet, populating the `matches` and `player_match_stats` tables automatically on file upload.

Milestone 3: Leaderboard & Chemistry Matrix UI Build (Weeks 5-6)
Construct the public-facing pages. Connect components to database views to display individual standings, calculated win rates, the interactive Chemistry grid matrix, and real-time Wacky Award callouts.

Milestone 4: Playoff Brackets & Tie-Breaker Logic Integration (Weeks 7-8)
Implement the custom playoff state engine. Code the Tier 1 Play-In bracket logic, the Pool-shuffling engine for Tier 2 group generation, and automatic RWR% tie-breaking filters to prepare for the final tournament phase.